

Mini E-commerce Distribuído: Relatório

Disciplina	Fundamentos de Computação Concorrente, Paralela e Distribuída
Projeto	Plataforma de e-commerce com API Gateway e três microsserviços
Stack	Node.js 18, Express, JWT, bcrypt js, arquivos JSON
Comunicação	HTTP/REST via proxy reverso no Gateway

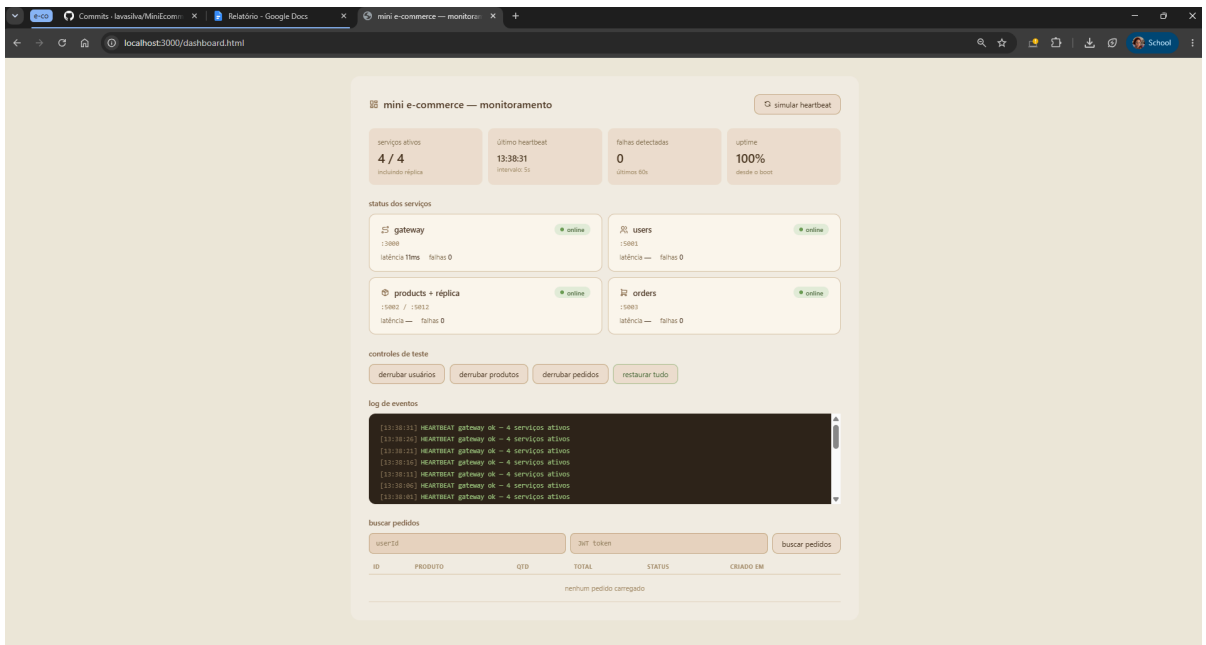
1. Comunicação entre os microsserviços

Optei por HTTP/REST como protocolo de comunicação por ser a opção mais direta para o escopo do projeto e por facilitar a validação manual dos endpoints, documentada no README_execucao.md. Toda comunicação passa pelo API Gateway, que funciona como proxy reverso: o cliente só conhece a porta 3000, e os serviços internos ficam inacessíveis diretamente.

O Gateway foi implementado com o módulo nativo http do Node.js, sem bibliotecas de proxy externas, para deixar explícita a lógica de repasse, inclusive do header Authorization com o token JWT, que é repassado intacto aos serviços internos para que cada um faça sua própria validação.

O serviço de Pedidos também faz uma chamada direta ao serviço de Produtos para validar se o produto existe antes de confirmar um pedido, o que é o único ponto de comunicação serviço a serviço fora do Gateway.

Serviço	Porta	Rotas expostas via Gateway
Usuários	:5001	POST /users/register, POST /users/login, GET /users/:id
Produtos	:5002 / :5012	GET /products, GET /products/:id, POST /products (admin)
Pedidos	:5003	POST /orders, GET /orders/:userId
Gateway	:3000	GET /health, GET /status (dashboard)



2. Replicação e estratégia de consistência

Para a replicação do serviço de Produtos, escolhi consistência forte. A lógica é simples: quando um produto é criado, o nó primário (porta 5002) persiste o dado localmente e só então propaga para a réplica (porta 5012) via POST /products/replicate. A resposta 201 ao cliente só é enviada depois que os dois nós confirmam a gravação.

Escolhi essa abordagem porque, para um e-commerce, a ideia de um cliente criar um produto e outra requisição imediata não encontrá-lo é inaceitável do ponto de vista de experiência. A consistência eventual teria menor latência de escrita, mas admitiria essa janela de inconsistência. Como o projeto é didático e o volume de requisições é baixo, a penalidade de latência da consistência forte é irrelevante.

Para leituras, implementei round-robin simples: cada GET alterna entre o primário e a réplica, distribuindo a carga.

Operação	Comportamento
Escrita (POST)	Grava no primário → propaga para réplica → confirma ao cliente
Leitura (GET)	Round-robin entre primário (:5002) e réplica (:5012)
Estratégia	Consistência forte — leitura sempre reflete a última escrita confirmada

```
PS C:\Users\LaviniaSilva\vscode\ecommerce> curl -s -X POST http://localhost:3000/products -H "Content-Type: application/json" -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQ6IjIOTFRkbnVmc2ZiO2YmZlTQ5NjMtOGRIYy03YWZlZWFiYVNmJyIjCjI1bWpCbC16ImFkbWUuQGVtYWlsLnNvbS1InjYjZGUiOiJhZG1pbHIsImhhdC16MTc0MTM2OTAwNmCwiZChwIjoxNzgzMzk3ODAwQ0F0.WcUjHtOkUTBwBKsgzgv7P1JOWfIXDnlkq3qZdH6jyhpM" -d '{"name": "Notebook Pro", "price": 4500, "stock": 10}'
```

```
{
  "id": "72d26865-e6d4-497f-827b-f3dd8e764e91",
  "name": "Notebook Pro",
  "description": "",
  "price": 4500,
  "stock": 10,
  "createdAt": "2026-06-13T16:51:02.575Z"
}
```

```
PS C:\Users\LaviniaSilva\vscode\ecommerce> curl -s http://localhost:5002/products
```

```
[{"id": "72d26865-e6d4-497f-827b-f3dd8e764e91", "name": "Notebook Pro", "description": "", "price": 4500, "stock": 10, "createdAt": "2026-06-13T16:51:02.575Z"}]
```

```
PS C:\Users\LaviniaSilva\vscode\ecommerce> curl -s http://localhost:5012/products
```

```
[{"id": "72d26865-e6d4-497f-827b-f3dd8e764e91", "name": "Notebook Pro", "description": "", "price": 4500, "stock": 10, "createdAt": "2026-06-13T16:51:02.575Z"}]
```

```
PS C:\Users\LaviniaSilva\vscode\ecommerce>
```

3. Tolerância a falhas: o que acontece se Pedidos cair?

O Gateway implementa heartbeat a cada 5 segundos: envia GET /health a cada serviço e, após 2 falhas consecutivas sem resposta, marca o serviço como indisponível e registra o evento em log com timestamp. A partir daí, qualquer requisição para aquele serviço recebe 503 Service Unavailable diretamente do Gateway, sem nem tentar o repasse.

Se o serviço de Pedidos cair, o sistema continua operando normalmente para tudo que não envolva pedidos: autenticação funciona, produtos podem ser listados e criados. Só as rotas /orders retornam 503. Quando o serviço volta, o Gateway detecta na próxima rodada de heartbeat e registra o RECOVERY em log, voltando a rotear normalmente.

Essa abordagem de falha parcial isolada foi intencional: preferi que o sistema degradasse de forma controlada a deixar requisições presas em timeout.

Evento	Comportamento do sistema
Pedidos indisponível	Usuários e Produtos seguem operando; /orders retorna 503
Gateway detecta queda	Log: [timestamp] FAILURE service=orders — após 2 tentativas
Serviço recuperado	Log: [timestamp] RECOVERY service=orders — roteamento normal

Para demonstrar a tolerância a falhas, encerrei manualmente o processo do serviço de Pedidos e aguardei aproximadamente 15 segundos. O Gateway detectou a ausência de resposta no /health após duas tentativas consecutivas e registrou o evento de FAILURE com timestamp. Em seguida, reiniciei o serviço e, no próximo ciclo de heartbeat (cerca de 5 segundos), o Gateway registrou automaticamente o RECOVERY sem nenhuma intervenção manual no roteamento.

```
[2026-06-13T16:55:48.678Z] FAILURE service=orders url=http://localhost:5003 attempts=2
[2026-06-13T17:01:34.287Z] RECOVERY service=orders url=http://localhost:5003
```

4. JWT e controle de acesso por papel

No login, o serviço de Usuários emite um JWT assinado com HS256 contendo `userId`, `email`, `role` e

exp (expiração de 8 horas). Usei bcryptjs com salt 10 para armazenar as senhas, o que é mais seguro que SHA-256 simples porque inclui salt automático e é computacionalmente mais custoso por design.

O campo role pode ser "user" ou "admin". Como ele está dentro do payload assinado com a chave secreta, o cliente não consegue alterá-lo sem invalidar a assinatura. O endpoint POST /products aplica dois middlewares em sequência: verifyToken (valida assinatura e expiração) e requireAdmin (verifica role === "admin"). Se o role for "user", a requisição é rejeitada com 403 Forbidden antes de qualquer lógica de negócio.

Escolhi validar o token em cada microserviço individualmente, e não só no Gateway, porque isso garante segurança mesmo que alguém acesse um serviço diretamente, sem passar pelo proxy.

Papel	O que pode fazer
admin	Criar produtos, autenticar, criar e ver pedidos próprios
user	Autenticar, listar produtos, criar e ver pedidos próprios; 403 em POST /products
sem token	401 Unauthorized em qualquer rota protegida

```
PS C:\Users\LaviniaSilva\vscode\ecommerce> curl -s -X POST http://localhost:3000/users/login -H "Content-Type: application/json" -d '{"email": "admin@email.com", "password": "admin123"}'
{"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJhOTFkdWZiO2YmQzLTQ5NiJtOGRIYy03YWZlZWJYbWVhYmMjYiLCJlbWVhYmFpbCI6ImFkbWluQGVtYWlsLmNvbnR5cCI6InJvbnVjbGU0Ij0iOiJhZGpbiIsImIhdCI6MTc4MTM2OTAwNCwiZXhwIjoxNzgzMzk3ODAwfQ.WcUjHToKUtBWBKszgv7P1JOWFLx0Nkq3qZdHn6jyhpM"}
PS C:\Users\LaviniaSilva\vscode\ecommerce>
```

```
PS C:\Users\LaviniaSilva\vscode\ecommerce> curl -s -X POST http://localhost:3000/users/register -H "Content-Type: application/json" -d '{"name": "Maria", "email": "maria@email.com", "password": "senha123"}'
{"id": "7c44f893-d263-48e3-890d-4a06812f3208", "name": "Maria", "email": "maria@email.com", "role": "user", "createdAt": "2026-06-13T16:54:04.596Z"}
PS C:\Users\LaviniaSilva\vscode\ecommerce> curl -s -X POST http://localhost:3000/users/login -H "Content-Type: application/json" -d '{"email": "maria@email.com", "password": "senha123"}'
{"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI3YzQ0Zjg5MjY1Kk4jYzLTQ4ZTMtODkwZC00YTA2ODIyZjMyMDg1LCJlbWVhYmFpbCI6ImFkbWluQGVtYWlsLmNvbnR5cCI6InJvbnVjbGU0Ij0iOiJhZGpbiIsImIhdCI6MTc4MTM2OTAwNCwiZXhwIjoxNzgzMzk3ODAwfQ.WcUjHToKUtBWBKszgv7P1JOWFLx0Nkq3qZdHn6jyhpM"}
PS C:\Users\LaviniaSilva\vscode\ecommerce> curl -s -X POST http://localhost:3000/products -H "Content-Type: application/json" -H "Authorization: Bearer TOKEN_DA_MARIA" -d '{"name": "Produto Proibido", "price": 99}'
{"error": "token inválido ou expirado"}
PS C:\Users\LaviniaSilva\vscode\ecommerce>
```

5. Limitações em relação a um sistema de produção

O projeto cumpre os objetivos didáticos, mas tenho consciência das suas limitações principais:

Área	Limitação	Impacto prático
Persistência	Arquivos JSON síncronos	Sem suporte a concorrência; risco de corrupção sob carga
Replicação	Sem rollback em falha parcial	Se a réplica falhar após escrita local, dados ficam inconsistentes
Gateway	Ponto único de falha (SPOF)	Queda do Gateway derruba todo o sistema
Segurança interna	HTTP entre serviços (sem TLS)	Tokens JWT trafegam em texto claro na rede interna
Chave JWT	Estática, sem rotação	Comprometimento da chave expõe todos os tokens ativos
Réplica offline	Sem ressincronização ao voltar	Réplica pode ficar desatualizada após período fora